IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS

INSTRUCTION ARCHITECTURES AND COMPILERS

Wednesday, 7 May 2025, 2:30 PM

Time allowed: 2:00 hours

There are EIGHT questions on this paper.

Answer ALL questions.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : P.Y.K. Cheung, J. Wickerson
Second Marker(s) : Aaron Zhao, P.Y.K. Cheung

© Imperial College London

ELEC50010 INSTRUCTION ARCHITECTURES AND COMPILERS

NB: Questions 1–5 relate to the Instruction Architectures half of the module. Questions 6–8 relate to the Compilers half of the module. A summary of the RISC-V RV32I instruction set is provided at the back of this paper.

1. Use the instruction set summary provided, translate the following RISC-V RV32I assembly language instructions into machine code instructions as 32-bit hexadecimal numbers.

   a) `and  a1, s6, s7` [4]

   b) `srli  a1, t1, 0x5` [4]

2. What are the assembly language instructions of the following RISC-V RV32I machine code instructions?

   a) `32'h00fffe17` [5]

   b) `32'h01e62523` [5]

3. Figure 3.1 shows the state diagram of a finite state machine (FSM) with two input signals `X` and `Y`, and one output signal `Z`, driven with a clock signal `CLK`. Changes in `X` and `Y` are synchronised on the falling edges of `CLK`; changes in state and `Z` are synchronised on the rising edges of `CLK`.

a) Complete the timing diagram shown in Figure 3.2 given that the FSM is initially in state A. [5]

b) Design the FSM in SystemVerilog HDL. [7]

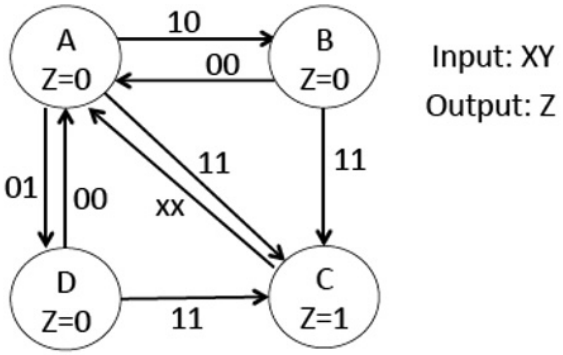


Figure 3.1

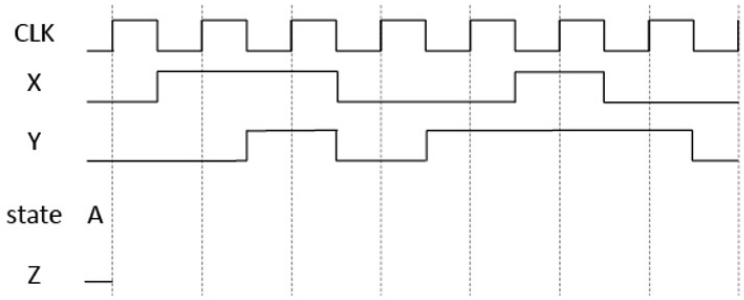


Figure 3.2

4. The branch (B-type) instructions for RV32I CPU has the instruction format shown in Figure 4.1.

a) How is the immediate operand from the B-type instruction used to compute the branch target address (BTA)? [3]

b) Design in SystemVerilog the module `B_type_imm()` as shown in Figure 4.2, which takes a 32-bit instruction code and provides as output the 32-bit immediate offset operand for use to compute the BTA. [3]

c) Design the program counter unit in the form of a schematic block diagram that produces the program counter of the CPU which implements the B-type instructions in addition to other regular instructions. (Do not implement any other instructions that change the control flow of the CPU except branch instructions.) [6]

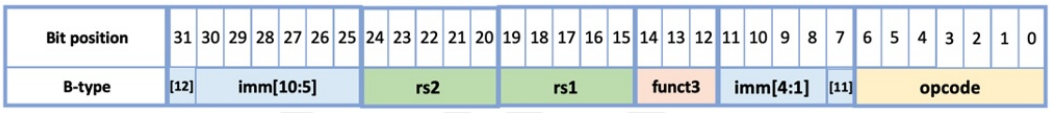


Figure 4.1

```
module B_type_imm(
    input   logic [31:0]    instr,      // 32-bit machine code instruction
    output  logic [31:0]    offset      // 32-bit branch offset
);
```

Figure 4.2

5. A computer system has a physical memory of 256 MB (2^{28} bytes), a virtual memory address space of 4GB (2^{32} bytes) and a page size of 8kB (2^{13} bytes).

a) State three advantages of using virtual memory instead of physical memory. [3]

b) Draw a diagram showing how the virtual address is divided into virtual page number (VPN) and page offset, and the physical address is divided into physical page number (PPN) and page offset. [3]

c) What is a Translation Lookaside Buffer (TLB)? [2]

6. Adrian is writing a parser for his new programming language. He must choose between a *top-down* parser and a *bottom-up* parser.

a) Briefly explain the difference between the two types of parser. [2]

b) Give one advantage of a top-down parser. [2]

c) Give one advantage of a bottom-up parser. [2]

7. Patricia is building a C compiler. She proposes to compile `switch` statements in two stages: by first converting them into `if` statements, and then compiling those `if` statements in the usual way. Her idea is to take C code of the form:

```
switch(e) {
  case v1: ss1
  case v2: ss2
  ...
  case vn: ssn
  default: ss
}
```
(before)

where e is a C expression, each v is a constant, and each ss is a sequence of C statements, and convert it into C code of the form:

```
if(e == v1) {
  ss1
}
else if (e == v2) {
  ss2
}
...
else if (e == vn) {
  ssn
}
else {
  ss
}
```
(after)

a) Write down a valid C function that will produce an *incorrect* result if it is compiled according to Patricia's proposal. Your function should be named `f`, it should take no parameters, and it should return an `int`. [6]

b) Fix Patricia's proposal by changing the form of the code labelled '(before)'. Leave the '(after)' code unchanged. [3]

c) Now setting aside any concerns about *correctness*, give two reasons why Patricia's proposal might lead to the compiler generating *inefficient* code. [9]

8. This question is about building a parser for the following context-free grammar over the alphabet $\{\mathtt{a}, \mathtt{b}, \mathtt{c}\}$:

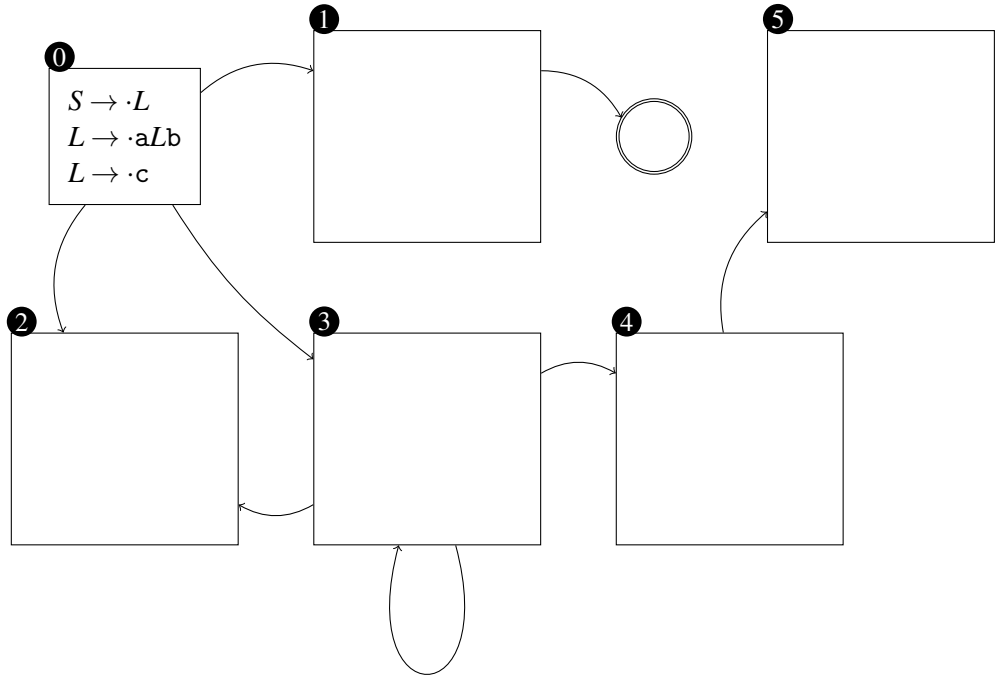
$$L ::= \mathtt{a}L\mathtt{b} \mid \mathtt{c}$$

a) Is L *nullable*? Briefly explain your answer. [2]

b) What is the *FIRST* set of L? Briefly explain your answer. [3]

c) What is the *FOLLOW* set of L? Briefly explain your answer. *[Hint: If you need it, you can use $ as a special symbol to denote 'end of input'.]* [3]

d) Suppose we decide to build a shift/reduce parser for L. Complete the following diagram by filling the five empty rectangles with item-sets and labelling the eight edges with symbols. The diagram is reproduced at the end of this paper so you can write on it, tear it off, and submit it with your answers. *[Hint: If you need it, you can use $ as a special symbol to denote 'end of input'.]* [9]



e) Using information from your answers to parts (a)–(d), complete the following parsing table for the shift/reduce parser. The table is reproduced at the end of this paper so you can write on it, tear it off, and submit it with your answers.

STATE	**ACTION**				**GOTO**
	`a`	`b`	`c`	$	*L*
❶					
❷					
❸					
❹					
❺					
❻					

[Hint: Each cell in the **ACTION** *columns should either:*

- *be left empty,*
- *contain 'accept',*
- *contain 'sN' where N is the state to enter after shifting, or*
- *contain 'r(foo→baz)' where 'foo→baz' is the grammar rule that should be reduced,*

and each cell in the **GOTO** *column should either:*

- *be left empty, or*
- *contain 'N' where N is the state to enter after reducing.]*

[9]

RISC-V Instruction Set Summary (1)

Register (R-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R-type	funct7							rs2					rs1					funct3			rd					opcode						

opcode	funct3	funct7	instruction	description	operation
7'h33	3'b000	7'h00	add rd, rs1, rs2	add	rd = rs1 + rs2
7'h33	3'b000	7'h20	sub rd, rs1, rs2	subtract	rd = rs1 - rs2
7'h33	3'b001	7'h00	sll rd, rs1, rs2	shift left logical	rd = rs1 << $rs2_{4:0}$
7'h33	3'b010	7'h00	slt rd, rs1, rs2	set less than	rd = (rs1 < rs2)
7'h33	3'b011	7'h00	sltu rd, rs1, rs2	set less than unsigned	rd = (rs1 < rs2)^{unsigned}
7'h33	3'b100	7'h00	xor rd, rs1, rs2	bit-wise XOR	rd = rs1 ^ rs2
7'h33	3'b101	7'h00	srl rd, rs1, rs2	shfit right logical	rd = rs1 >> $rs2_{4:0}$
7'h33	3'b101	7'h20	sra rd, rs1, rs2	shfit right arithmetic	rd = rs1 >>> $rs2_{4:0}$
7'h33	3'b110	7'h00	or rd, rs1, rs2	bit-wise OR	rd = rs1 | rs2
7'h33	3'b111	7'h00	and rd, rs1, rs2	bit-wise AND	rd = rs1 & rs2

Immediate (I-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
I-type	imm[11:0]												rs1					funct3			rd					opcode						

opcode	funct3	imm[11:5]	instruction	description	operation
7'h03	3'b000	-	`lb rd, imm(rs1)`	load byte immediate	rd = sign_ext ([address]$_{7:0}$)
7'h03	3'b001	-	`lh rd, imm(rs1)`	load half immediate	rd = sign_ext ([address]$_{15:0}$)
7'h03	3'b010	-	`lw rd, imm(rs1)`	load word immediate	rd = [address]$_{31:0}$
7'h03	3'b100	-	`lbu rd, imm(rs1)`	load byte unsigned	rd = zerp_ext ([address]$_{7:0}$)
7'h03	3'b101	-	`lhu rd, imm(rs1)`	load half unsigned	rd = zero_ext ([address]$_{15:0}$)
7'h13	3'b000	-	`addi rd, rs1, imm`	add immediate	rd = rs1 + sign_ext (imm)
7'h13	3'b001	7'h00	`slli rd, rs1, uimm`	Shift left logical immediate	rd = rs1 << uimm
7'h13	3'b010	-	`slti rd, rs1, imm`	set less than immediate	rd = (rs1 < sign_ext(imm))
7'h13	3'b011	-	`sltiu rd, rs1, uimm`	set less than imm. unsigned	rd = (rs1 < zero_ext(imm))
7'h13	3'b100	-	`xori rd, rs1, imm`	bit-wise XOR immediate	rd = rs1 ^ sign_ext (imm)
7'h13	3'b101	7'h00	`srli rd, rs1, uimm`	Shift right logical immediate	rd = rs1 >> uimm
7'h13	3'b101	7'h20	`srai rd, rs1, uimm`	Shift right arithmetic imm.	rd = rs1 >> uimm
7'h13	3'b110	-	`ori rd, rs1, imm`	bit-wise OR immediate	rd = rs1 | sign_ext (imm)
7'h13	3'b111	-	`andi rd, rs1, imm`	bit-wise AND immediate	rd = rs1 & sign_ext (imm)

Upper Immediate (U-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
U-type	imm[31:12]																				rd					opcode						

opcode	instruction	description	operation
7'h17	`auipc rd, upimm`	add upper immediate to PC	rd = {upimm, 12'b0} + PC
7'h37	`lui rd, upimm`	load upper immediate	rd = {upimm, 12'b0}

RISC-V Instruction Set Summary (2)

Store (S-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
S-type	imm[11:5]							rs2					rs1					funct3			imm[4:0]					opcode						

opcode	funct3	instruction	description	operation
7'h23	3'b000	`sb rs2, imm(rs1)`	store byte immediate	$[address]_{7:0} = rs2_{7:0}$
7'h23	3'b001	`sh rs2, imm(rs1)`	store half immediate	$[address]_{15:0} = rs2_{15:0}$
7'h23	3'b010	`sw rs2, imm(rs1)`	store word immediate	$[address]_{31:0} = rs2$

Branch (B-type) Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
B-type	[12]	imm[10:5]						rs2					rs1					funct3			imm[4:1]				[11]	opcode						

opcode	funct3	instruction	description	operation
7'h63	3'b000	`beq rs1, rs2, label`	branch if equal	If (rs1 == rs2) PC = BTA
7'h63	3'b001	`bne rs1, rs2, label`	branch if not equal	If (rs1 ≠ rs2) PC = BTA
7'h63	3'b100	`blt rs1, rs2, label`	branch if less than	If (rs1 < rs2) PC = BTA
7'h63	3'b101	`bge rs1, rs2, label`	branch if greater than or equal	If (rs1 ≥ rs2) PC = BTA
7'h63	3'b110	`bltu rs1, rs2, label`	branch if less than unsigned	If (rs1 < rs2) PC = BTA
7'h63	3'b111	`bgeu rs1, rs2, label`	branch if greater or equal unsigned	If (rs1 ≥ rs2) PC = BTA

BTA = Branch Target Address, PC = Program Counter

Jump-and-link Instructions

Bit position	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
J-type	[20]	imm[10:1]										[11]	imm[19:12]								rd					opcode						
jalr – I-type	imm[11:0]												rs1					3'b000			rd					opcode						

opcode	instruction	description	operation
7'h67	`jalr rd, rs1, imm`	jump and link register	PC = rs1 + sign_ext(imm), rd = PC + 4
7'h6f	`jal rd, label`	Jump and link	PC = JTA, rd = PC + 4

JTA = Jump Target Address, PC = Program Counter

Register names

name	register number	usage
zero	x0	Constant value 0
ra	x1	Return Addrss
sp	x2	Stack Pointer
gp	x3	Global Pointer
tp	X4	Thread Pointer
t0 – t2	x5 – x7	Temporary registers
s0/fp	x8	Saved register/Frame Pointer
s1	x9	Saved register
a0 – a1	x10 – x11	Function arguments/ return values
a2 – a7	x12 – x17	Function arguments
s2 – s11	x18 – x27	Saved registers
t3 – t6	x28 – x31	Temporary registers

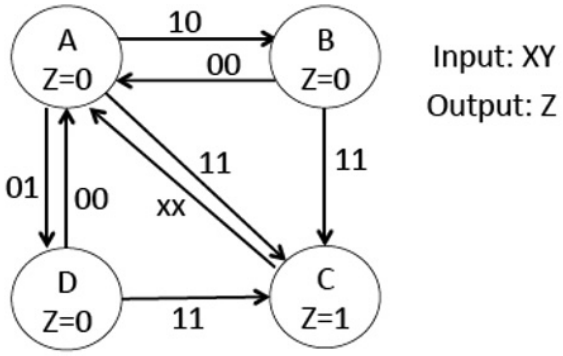
Abbreviations

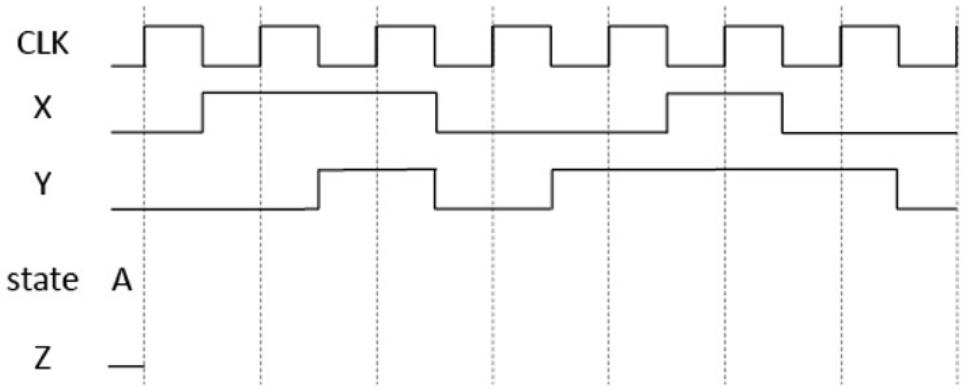
imm	12-bit signed immediate ($imm_{11:0}$)
uimm	5-bit unsigned immediate ($imm_{4:0}$)
upimm	20-bit upper immediate ($imm_{31:12}$)
address	memory addres: rs1 + sign_ext($imm_{11:0}$)
[address]	data at memory location address
BTA	branch target address: PC + sign_ext({$imm_{12:1}$, 1'b0})
JTA	jump target address: PC + sign_ext({$imm_{20:1}$, 1'b0})
label	text label indicating instruction address
sign_ext	value sign-extended to 32 bits
zero_ext	value zero-extended to 32 bits

SUPPLEMENTARY SHEET FOR QUESTION 3

You may tear off this page, annotate it with your answer, and attach it to the answer book.

Diagram for question 3a:





SUPPLEMENTARY SHEET FOR QUESTION 8

You may tear off this page, annotate it with your answer, and attach it to the answer book.

Diagram for question 8d:

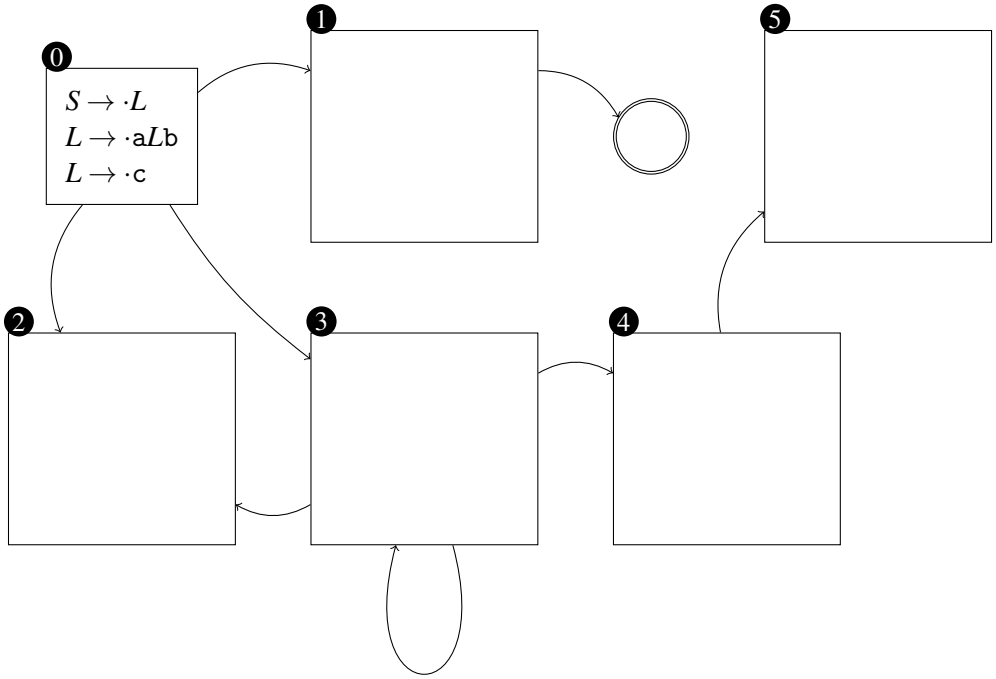


Table for question 8e:

STATE	**ACTION**				**GOTO**
	a	b	c	$	L
0					
1					
2					
3					
4					
5					